Análise Comparativa de Estruturas Disjoint Set Union: Impacto das Heurísticas de Otimização no Algoritmo de Kruskal

Enzo Barcelos[1], **Luiz Aldenucci**[2], **Arthur Sita**[3], **Rafael Ferraz**[4]

[1]Curso de Engenharia de Software – Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

enzo.barcelos@sga.pucminas.br[1], laldenucci@sga.pucminas.br[2]

1388895@sga.pucminas.br[3], 1384417@sga.pucminas.br[4]

Resumo. *Este trabalho analisa experimentalmente três variantes da estrutura Disjoint Set Union (DSU) aplicadas ao algoritmo de Kruskal para o cálculo de Árvores Geradoras Mínimas: a implementação Ingênua, Union by Rank e a versão completa de Tarjan (Union by Rank com Path Compression). Os experimentos foram realizados em grafos conexos aleatórios com* n *entre* 10^3 *e* 10^5 *vértices e* $m = 3n$ *arestas, medindo tempo de execução e número de acessos ao array de pais. As medições reportadas são médias sobre cinco grafos distintos por configuração, acompanhadas do desvio padrão amostral nos tempos. Os resultados evidenciam na prática a transição entre as classes* $O(n)$*,* $O(\log n)$ *e* $O(\alpha(n))$*: a versão Ingênua ultrapassa* 8×10^9 *acessos e aproximadamente 16 segundos em* $n = 10^5$*, enquanto Tarjan mantém o custo médio por aresta essencialmente constante, com redução de mais de três ordens de grandeza. Com a estrutura de Tarjan, o custo do DSU deixa de influenciar o tempo total do Kruskal, que passa a ser dominado pela ordenação das arestas.*

Abstract. *This paper presents an experimental analysis of three Disjoint Set Union (DSU) variants applied to Kruskal's Minimum Spanning Tree algorithm: Naive, Union by Rank, and Full Tarjan (Union by Rank combined with Path Compression). Experiments were conducted on randomly generated connected graphs with* n *ranging from* 10^3 *to* 10^5 *vertices and* $m = 3n$ *edges, measuring both execution time and array-access counts. Each data point is the mean over five distinct graphs, with sample standard deviation reported for timings. Results show a clear practical transition across complexity classes* $O(n)$*,* $O(\log n)$*, and* $O(\alpha(n))$*: the Naive implementation reaches over* 8×10^9 *accesses and around 16 seconds at* $n = 10^5$*, while Tarjan keeps the per-edge cost essentially constant, reducing accesses by more than three orders of magnitude. Under Tarjan's structure, the dominant cost in Kruskal shifts entirely to edge sorting, confirming that the DSU overhead becomes negligible in practice.*

Palavras-chave: Union-Find; Disjoint Set Union; Algoritmo de Kruskal; Path Compression; Complexidade Amortizada; Função Inversa de Ackermann.

1. Introdução

Gerenciar conjuntos disjuntos é um problema recorrente em computação. Aparece no processamento de imagens, quando pixels vizinhos precisam ser agrupados em regiões.

Aparece em redes, quando se quer saber se dois nós pertencem à mesma componente conectada. E aparece, talvez da forma mais explícita, em algoritmos de grafos que testam conectividade dinamicamente.

A estrutura de dados conhecida como *Disjoint Set Union* (DSU), ou simplesmente *Union-Find*, foi projetada para resolver esse problema de forma eficiente. Ela mantém uma coleção de conjuntos disjuntos e oferece duas operações fundamentais: `Find`, que identifica a qual conjunto um elemento pertence, e `Union`, que funde dois conjuntos em um só.

Uma implementação simples dessa estrutura, utilizando árvores sem nenhum critério de balanceamento, pode levar a cadeias longas de ponteiros, degradando o desempenho das operações para $O(n)$ no pior caso. Robert Tarjan propôs duas heurísticas que, quando combinadas, reduzem a complexidade amortizada para $O(\alpha(n))$, onde α é a inversa da função de Ackermann. Essa função cresce tão lentamente que, para qualquer valor prático de n, seu resultado não ultrapassa 4 [1].

O presente trabalho implementa três variantes do DSU com níveis crescentes de otimização e avalia experimentalmente o impacto no desempenho do algoritmo de Kruskal para Árvores Geradoras Mínimas (MST). Duas questões orientam a análise: (i) a partir de qual escala a implementação ingênua deixa de ser viável em aplicações reais de MST? e (ii) qual o ganho marginal efetivo da combinação de Union by Rank com Path Compression em relação ao uso isolado de Union by Rank?

A contribuição do trabalho é empírica: as três variantes foram implementadas em Java com instrumentação de contadores de acesso a memória, e os resultados são apresentados tanto em escala linear quanto logarítmica, incluindo uma análise de expoente empírico que quantifica as classes de complexidade observadas. O Algoritmo de Kruskal foi escolhido como ambiente de teste por exercitar, por construção, sequências intercaladas de `Find` e `Union` na proporção $\Theta(m)$, cenário que expõe com clareza as diferenças entre as variantes.

O restante do artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica; a Seção 3 descreve a metodologia e o ambiente experimental; a Seção 4 discute os resultados obtidos; e a Seção 5 conclui o trabalho.

2. Fundamentação Teórica

2.1. A Estrutura Disjoint Set Union

A estrutura DSU representa cada conjunto como uma árvore enraizada. Cada elemento possui um ponteiro para seu pai, e a raiz de cada árvore serve como representante do conjunto. Inicialmente, cada elemento é seu próprio pai, formando n conjuntos unitários.

A operação `Find(x)` percorre a cadeia de ponteiros a partir de x até encontrar a raiz, ou seja, o elemento cuja referência aponta para si mesmo. A operação `Union(x, y)` localiza as raízes dos conjuntos de x e y e, caso sejam diferentes, faz a raiz de um apontar para a raiz do outro.

2.2. Implementação Ingênua

Na versão mais simples, o `Union` conecta uma raiz à outra sem nenhum critério de escolha. O problema dessa abordagem é que, em sequências adversas de operações, a árvore

pode degenerar em uma lista encadeada. Nesse cenário, cada chamada a `Find` precisa percorrer até n ponteiros, resultando em complexidade $O(n)$ por operação.

2.3. Union by Rank

A heurística de *Union by Rank* busca manter as árvores balanceadas. Cada nó mantém um valor de *rank*, que funciona como um limitante superior da altura da subárvore. Na união de dois conjuntos, a raiz com menor *rank* é pendurada sob a raiz com maior *rank*. Quando ambos têm o mesmo *rank*, escolhe-se uma arbitrariamente e incrementa-se seu *rank*.

Essa estratégia garante que a altura de qualquer árvore com k elementos não ultrapasse $\lfloor \log_2 k \rfloor$. Isso ocorre porque, para que uma árvore atinja *rank* r, ela precisa ter sido unida com outra árvore de *rank* $r-1$, o que exige ao menos 2^r elementos. Portanto, a complexidade do `Find` fica limitada a $O(\log n)$.

2.4. Path Compression

A heurística de *Path Compression* modifica a operação `Find` para que, ao percorrer o caminho até a raiz, todos os nós visitados passem a apontar diretamente para ela. Isso "achata" a árvore progressivamente, tornando consultas futuras sobre os mesmos elementos praticamente instantâneas.

Usada isoladamente, sem Union by Rank, Path Compression garante complexidade amortizada $O(\log n)$ por operação [1], com pior caso $O(n)$ em chamadas individuais. O ganho expressivo vem justamente da combinação: árvores com altura limitada por $O(\log n)$ (via Rank) e compressão progressiva desse caminho curto (via PC) resulta na complexidade amortizada $O(\alpha(n))$, que é o tema da seção seguinte.

2.5. A Complexidade $O(\alpha(n))$ de Tarjan

Para entender por que a combinação de Union by Rank e Path Compression produz complexidade amortizada $O(\alpha(n))$, é preciso distinguir custo real de custo amortizado. A análise amortizada distribui o custo total de uma sequência de operações uniformemente entre elas: mesmo que uma operação individual seja cara, o custo médio por operação ao longo da sequência pode ser muito menor.

Tarjan provou esse resultado usando o *método do potencial* [3]: associa-se a cada nó uma função de potencial que depende do seu rank e de sua posição na árvore. Operações de `Find` que percorrem caminhos longos reduzem o potencial ao comprimir nós, de modo que seu custo real alto é compensado por uma queda equivalente no potencial. O custo amortizado por operação (custo real somado à variação de potencial) fica limitado a $O(\alpha(n))$.

O resultado formal é o seguinte: qualquer sequência de m operações sobre n elementos tem custo total $O((m+n)\cdot\alpha(n))$, onde α é a inversa da função de Ackermann [1, 2].

A função de Ackermann $A(k,j)$ é definida recursivamente como [5]:

$$A(1,j) = 2^j, \quad \text{para } j \geq 1 \tag{1}$$

$$A(k,1) = A(k-1,2), \quad \text{para } k \geq 2 \tag{2}$$

$$A(k,j) = A(k-1, A(k,j-1)), \quad \text{para } k \geq 2, j \geq 2 \tag{3}$$

Essa função cresce de forma super-exponencial. $A(4,1)$, por exemplo, supera uma torre de potências de 2 com $2^{16} = 65.536$ andares. Sua inversa $\alpha(n) = \min\{k \geq 1 : A(k,1) \geq n\}$ cresce com a mesma lentidão, agora para o lado oposto. Para qualquer n imaginável no universo físico, $\alpha(n) \leq 4$ [5]. Formalmente a complexidade não é $O(1)$; na prática, é constante.

A Tabela 1 ilustra essa propriedade com valores concretos.

Tabela 1. Valores de $\alpha(n)$ para diferentes ordens de grandeza de n

n	$\alpha(n)$
≤ 2	1
$\leq 2^4 = 16$	2
$\leq 2^{65536}$	3
$\leq A(4,1)$ (número com 10^{19728} dígitos)	4

2.6. O Algoritmo de Kruskal

O algoritmo de Kruskal resolve o problema da Árvore Geradora Mínima (MST) para grafos conexos ponderados [4]. Seu funcionamento é direto: ordena todas as arestas do grafo por peso crescente e percorre essa lista, adicionando cada aresta à MST se ela não formar ciclo com as arestas já selecionadas [6]. A detecção de ciclos é feita verificando se os dois vértices da aresta pertencem ao mesmo conjunto, o que é exatamente o que o DSU oferece.

A complexidade total do Kruskal é $O(m \log m)$ para a ordenação das arestas somada ao custo das operações de `Find` e `Union`. Com o DSU de Tarjan, essas operações têm custo amortizado $O(\alpha(n))$ cada, totalizando $O(m \cdot \alpha(n))$ para a parte de conjuntos disjuntos, o que na prática não adiciona overhead significativo além da ordenação.

3. Metodologia

3.1. Escolhas de Modularização

O projeto foi desenvolvido em Java e organizado em três pacotes, cada um com responsabilidade bem definida:

- **Pacote `dsu`**: Contém uma interface `DSU` que define o contrato das operações (`find`, `union`, `makeSet`) e um método `getContador()` para contagem de acessos. As três variantes (`DSUNaive`, `DSURank` e `DSUTarjan`) implementam essa interface.
- **Pacote `grafo`**: Contém a classe `Aresta` (com atributos `u`, `v`, `peso` e implementação de `Comparable`) e a classe `Kruskal`, que recebe qualquer objeto do tipo `DSU` por polimorfismo.
- **Pacote `benchmark`**: Contém a classe responsável por gerar grafos aleatórios, executar o Kruskal com cada variante e medir os resultados.

A vantagem dessa divisão é prática: uma nova variante de DSU entra no experimento criando-se uma classe que implemente a interface e passando-a ao Kruskal. Nenhum outro arquivo precisa mudar. Como consequência secundária, grafo e DSU podem ser testados em isolamento.

3.2. Implementação das Variantes

Cada variante do DSU mantém um contador interno que registra o total de acessos ao array `pai[]`: cada leitura usada para comparação ou avanço de ponteiro conta como 1 acesso; cada escrita (union ou compressão de caminho) também conta como 1. A política é uniforme entre as três variantes, conforme documentado na interface `DSU.java`, o que permite comparação direta independente de variações de tempo causadas pelo sistema operacional ou escalonamento de processos.

- **DSUNaive**: Utiliza apenas o array `pai[]`. O `find` percorre a cadeia até a raiz sem modificá-la. O `union` simplesmente conecta uma raiz à outra.
- **DSURank**: Adiciona o array `rank[]`. O `find` permanece sem compressão. O `union` compara os *ranks* e pendura a árvore menor sob a maior.
- **DSUTarjan**: Combina `rank[]` com compressão de caminho no `find`. A implementação é recursiva: cada chamada faz o nó visitado apontar diretamente para a raiz retornada.

3.3. Validação de Correção

Antes dos experimentos de desempenho, verificou-se que as três variantes produzem exatamente o mesmo peso total de MST para cada grafo de teste. O método `Main.exemploKruskal()` executa essa verificação num grafo de 6 vértices com solução conhecida e lança `AssertionError` caso qualquer variante divirja das demais (tolerância de 10^{-9}). A consistência dos pesos confirma que as otimizações não alteram a correção do algoritmo.

3.4. Geração de Grafos de Teste

Para cada tamanho n, foram gerados grafos com $m = 3n$ arestas. A geração garante conectividade construindo primeiro uma árvore geradora (cada vértice i é conectado a um vértice aleatório $j < i$) e então adicionando $m - (n - 1)$ arestas extras aleatórias. Os pesos são valores reais uniformemente distribuídos no intervalo $[0, 100)$.

Em cada configuração foram usadas cinco seeds fixas (de 42 a 46), gerando cinco grafos distintos. Dentro de um mesmo grafo, as três variantes processam exatamente a mesma sequência de arestas, de modo que a comparação de contagens e tempos é sempre feita sobre dados idênticos. A ordenação das arestas é feita por `Collections.sort()`, que usa o algoritmo Timsort com complexidade $O(m \log m)$.

Algumas limitações do gerador merecem registro: self-loops ($u = v$) são descartados sem reposição, de modo que o número efetivo de arestas pode ser levemente inferior a $3n$; arestas duplicadas são permitidas; e a topologia do grafo é enviesada para o padrão "árvore-caminho com arestas extras", não representando a distribuição uniforme de um grafo de Erdős–Rényi.

3.5. Cenários de Teste

Os testes foram realizados para cinco tamanhos de entrada: $n \in \{1.000;\ 5.000;\ 10.000;\ 50.000;\ 100.000\}$. Para cada configuração, o Kruskal foi executado com as três variantes de DSU em cinco grafos distintos, gerados com as seeds 42, 43, 44, 45 e 46. Em cada execução foram medidos o tempo via `System.nanoTime()` e a contagem de acessos ao array de pais. Os valores reportados nas tabelas são a média das cinco repetições; para os tempos, também se reporta o desvio padrão amostral.

3.6. Ambiente de Execução

Os experimentos foram conduzidos no seguinte ambiente:

- **Sistema Operacional**: Windows 11 Pro (Build 26200)
- **Processador**: Intel Core i7-13650HX (16 núcleos, clock base 2,6 GHz, boost até 4,9 GHz)
- **Memória RAM**: 40 GB dual-channel (32 GB DDR5-4800 + 8 GB DDR5-5600)
- **JVM**: Java SE 24, HotSpot 64-Bit Server VM (build 24+36-3646)
- **Compilador**: `javac 24`; sem flags de otimização adicionais

Antes de cada rodada de medição, uma fase de *warm-up* executa as três variantes em um grafo auxiliar de $n = 5.000$ vértices (descartado). Isso permite que o compilador JIT compile os caminhos quentes antes das medições oficiais, reduzindo o impacto da compilação em tempo de execução nos tempos registrados. Cada ponto das tabelas é a média de cinco execuções em grafos diferentes, cujas seeds de geração vão de 42 a 46. O desvio padrão reportado para os tempos usa o estimador amostral, com $n - 1$ no denominador.

4. Discussão de Resultados

4.1. Operações de Acesso à Memória

A Tabela 2 apresenta a contagem de acessos ao array `pai[]` para cada variante e tamanho de entrada.

Tabela 2. Número médio de operações de acesso ao array de pais (média sobre 5 seeds)

n	Naive	Union by Rank	Tarjan
1.000	791.283	29.540	20.682
5.000	20.144.948	154.647	106.589
10.000	80.860.069	321.354	215.950
50.000	2.040.461.352	1.629.513	1.079.521
100.000	8.199.815.162	3.283.355	2.166.669

A versão ingênua apresenta crescimento claramente superlinear: multiplicando n por 10 (de 10.000 para 100.000), o número de acessos cresce por um fator de aproximadamente 101. Esse valor é compatível com comportamento quadrático total, já que custo $O(n)$ por operação acumulado sobre $m = 3n$ operações resulta em $O(n^2)$ acessos. Union by Rank e Tarjan, em contraste, crescem de forma muito mais contida, com comportamento aproximadamente linear em n. É o reflexo esperado de operações com custo sublinear acumuladas sobre $\Theta(m) = \Theta(n)$ chamadas. O expoente empírico de crescimento, apresentado mais adiante, quantifica essa diferença de forma direta.

A Figura 1 ilustra a disparidade absoluta entre as variantes. A curva do Naive domina completamente o gráfico, enquanto Rank e Tarjan ficam próximas do eixo horizontal na mesma escala. Em $n = 10^5$, o Naive realiza cerca de 3.800 vezes mais acessos que Tarjan.

A Figura 2 faz o zoom entre Rank e Tarjan. A compressão de caminho reduz o número de acessos em cerca de 34% de forma consistente em todas as entradas testadas.

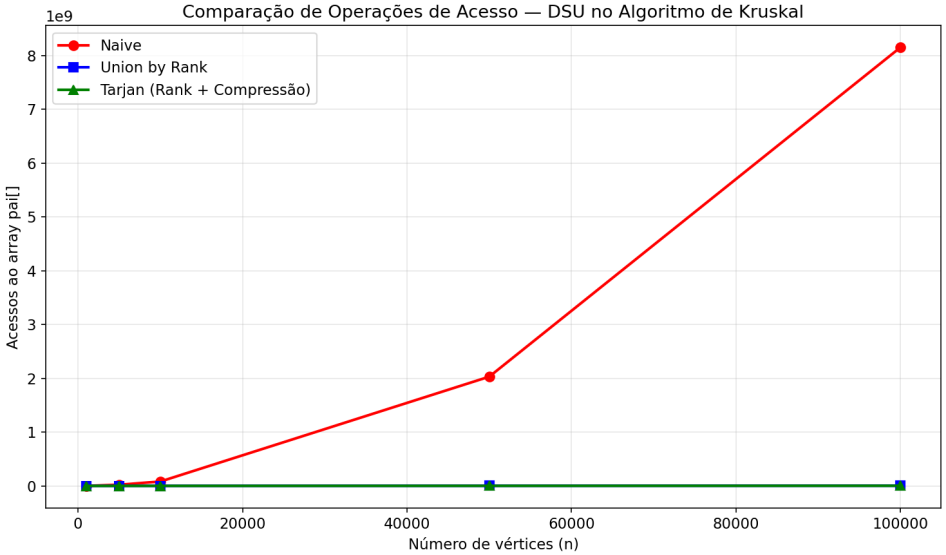

Figura 1. Comparação do número de operações entre as três variantes.

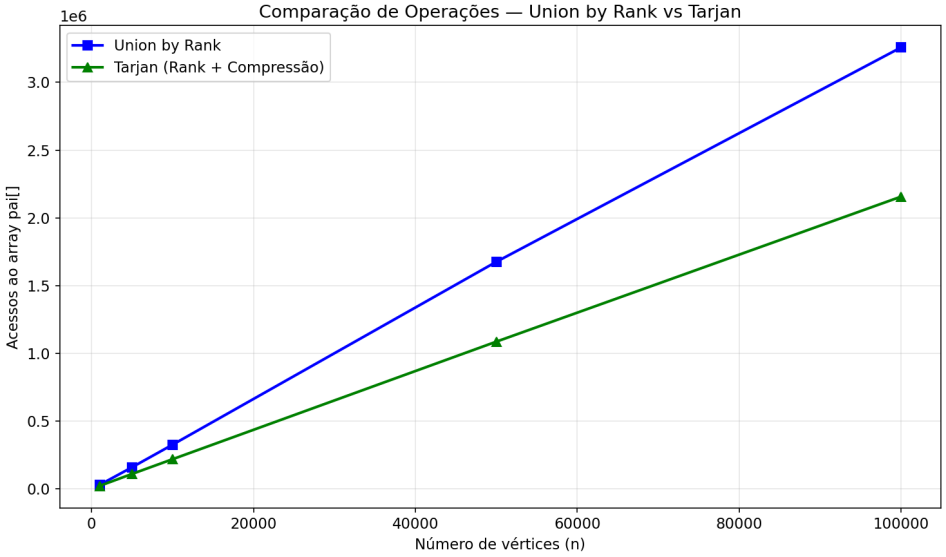

Figura 2. Comparação de operações entre Union by Rank e Tarjan.

4.2. Tempo de Execução

A Tabela 3 apresenta os tempos de execução medidos.

Os tempos de execução, medidos após a fase de *warm-up* do JIT, seguem a mesma hierarquia das contagens de acesso. Em $n = 10^5$, a versão Naive chega perto de 16,2 segundos, enquanto Rank e Tarjan ficam abaixo de 70 ms. A diferença passa de 230 vezes. Tarjan é mais rápido que Rank em todos os pontos testados; a vantagem é consistente, embora pequena em termos absolutos.

A Figura 3 evidencia essa disparidade em escala linear. A curva do Naive cresce de forma abrupta após $n = 10^4$, enquanto Rank e Tarjan permanecem praticamente sobrepostas à base do gráfico.

Tabela 3. Tempo de execução em milissegundos (média $\pm$ desvio padrão amostral sobre 5 seeds)

n	Naive (ms)	Rank (ms)	Tarjan (ms)
1.000	1,28 $\pm$ 0,07	0,68 $\pm$ 0,11	0,58 $\pm$ 0,04
5.000	13,94 $\pm$ 0,64	2,59 $\pm$ 0,10	2,39 $\pm$ 0,08
10.000	50,92 $\pm$ 1,16	5,47 $\pm$ 0,14	5,04 $\pm$ 0,18
50.000	3.392,31 $\pm$ 127,89	31,36 $\pm$ 2,59	29,07 $\pm$ 0,52
100.000	16.235,40 $\pm$ 1.662,05	68,60 $\pm$ 2,73	63,17 $\pm$ 1,19

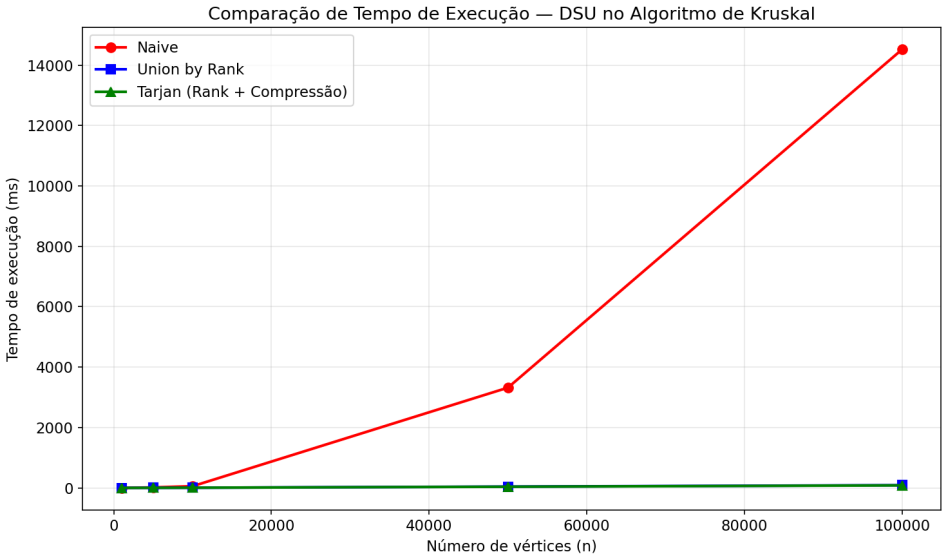

Figura 3. Comparação de tempo de execução entre as três variantes.

Tabela 4. Expoente empírico de crescimento das operações entre pares consecutivos de n (médias sobre 5 seeds)

Transição	Naive (p)	Rank (p)	Tarjan (p)
$10^3 \to 5 \cdot 10^3$	2,01	1,03	1,02
$5 \cdot 10^3 \to 10^4$	2,01	1,06	1,02
$10^4 \to 5 \cdot 10^4$	2,01	1,01	1,00
$5 \cdot 10^4 \to 10^5$	2,01	1,01	1,00

Para isolar a diferença entre as duas variantes otimizadas, a Figura 4 apresenta apenas Rank e Tarjan. Tarjan mantém vantagem consistente em todos os pontos, ainda que o ganho absoluto sobre Rank seja pequeno quando se considera o desvio padrão amostral.

4.3. Classes de Complexidade e Custo Amortizado

Para quantificar as classes de complexidade observadas experimentalmente, calcula-se o expoente empírico p entre pares consecutivos de n, definido como $p = \log(\text{ops}_{i+1}/\text{ops}_i)/\log(n_{i+1}/n_i)$. Se o número de operações cresce como n^p, o valor de p aproxima o expoente da classe de complexidade real. A Tabela 4 apresenta os valores calculados.

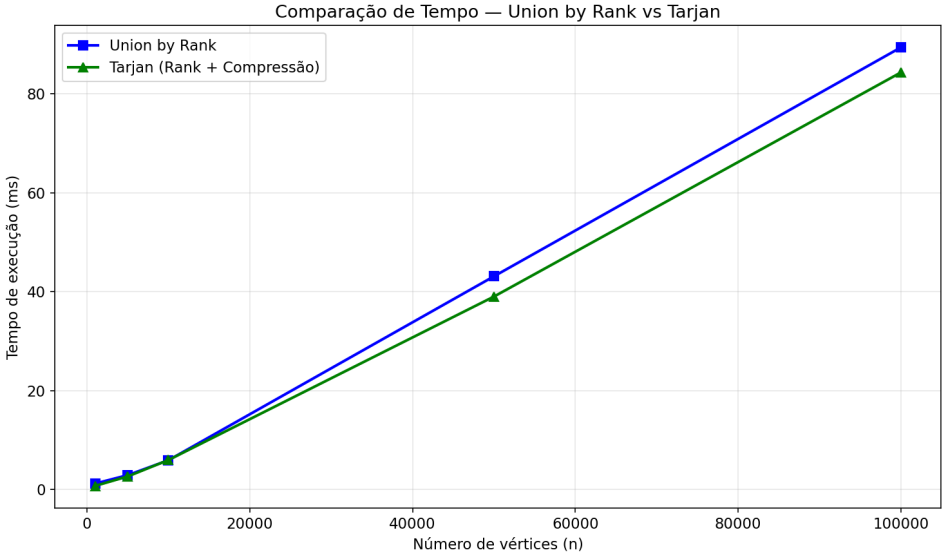

Figura 4. Comparação de tempo entre Union by Rank e Tarjan.

O expoente do Naive se mantém em torno de 2 em todas as transições, confirmando o crescimento quadrático $O(n^2)$. Rank e Tarjan têm expoentes próximos de 1, o que é esperado: como $m = 3n$, o custo total de $O(m \log n)$ para Rank e $O(m \cdot \alpha(n))$ para Tarjan cresce essencialmente de forma linear em n, com o fator logarítmico ou de Ackermann contribuindo marginalmente. A Figura 5 visualiza esse resultado.

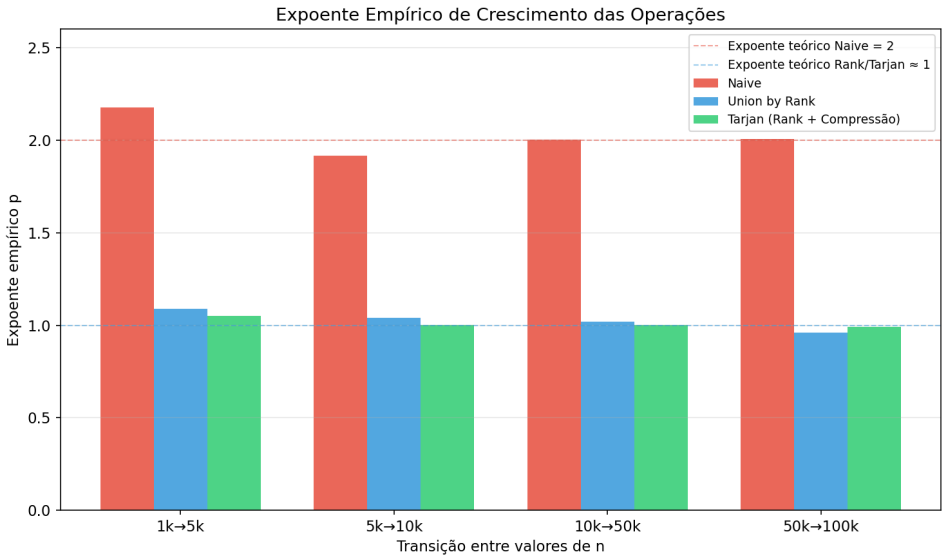

Figura 5. Expoente empírico de crescimento das operações por variante. Naive converge para 2 (quadrático); Rank e Tarjan ficam próximos de 1 (linear em n).

A Figura 6 usa escala logarítmica no eixo vertical. Nessa escala, curvas de classes diferentes exibem inclinações distintas, o que permite distinguir visualmente $O(n^2)$, $O(n \log n)$ e $O(n \cdot \alpha(n))$ sem ambiguidade. As três curvas têm inclinações claramente separadas, confirmando que as diferenças não são apenas de constante multiplicativa, mas

de classe assintótica.

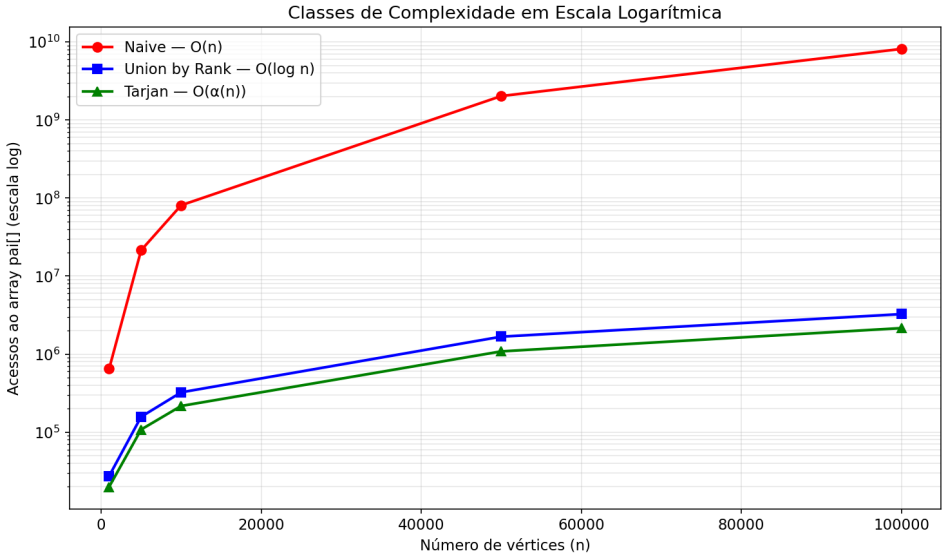

Figura 6. Operações em escala logarítmica — separação entre as classes de complexidade.

A Figura 7 mostra o custo médio por aresta. No Naive, esse custo cresce linearmente com n, como seria de se esperar de operações $O(n)$. Rank apresenta crescimento lento, compatível com uma dependência logarítmica. Tarjan é o caso mais interessante: o custo por aresta fica praticamente plano ao longo de duas ordens de grandeza, e é essa estabilidade que corresponde, empiricamente, à complexidade amortizada $O(\alpha(n))$.

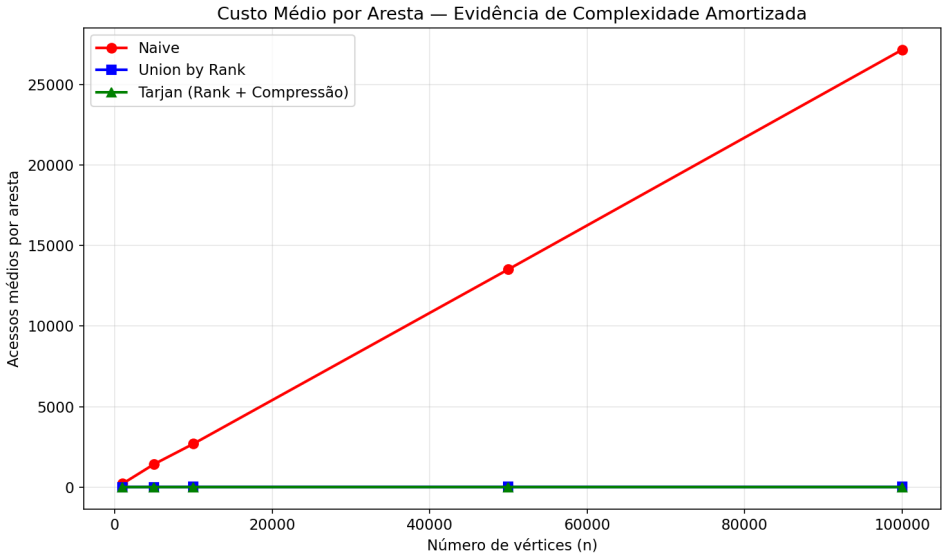

Figura 7. Custo médio de operações por aresta — evidência de complexidade amortizada.

5. Conclusão

A primeira pergunta da introdução era em que escala a implementação ingênua deixa de ser viável. Os experimentos respondem de forma bastante direta: já em $n = 10.000$, a

versão Naive gasta perto de 51 ms, enquanto Rank e Tarjan ficam abaixo de 5,5 ms. Em $n = 10^5$, a diferença passa de 230 vezes no tempo e de 3.800 vezes no número de acessos. Para qualquer aplicação de MST em escala real, Naive é simplesmente inviável.

A segunda pergunta mirava o ganho marginal de acrescentar Path Compression ao Union by Rank. O resultado é positivo, mas mais modesto do que o passo anterior: a compressão reduz os acessos em cerca de 34% e o tempo em torno de 8% em $n = 10^5$. É suficiente para manter Tarjan consistentemente à frente de Rank, mas a conquista principal já havia sido feita pelo Union by Rank. Na prática, o que importa mais é que, com a estrutura completa de Tarjan, o custo do DSU deixa de dominar o Kruskal: o gargalo passa a ser a ordenação das arestas em $O(m \log m)$.

Limitações

A topologia dos grafos gerados (árvore-caminho com arestas extras) não corresponde à distribuição uniforme de um grafo de Erdős–Rényi, o que pode influenciar a profundidade média das árvores DSU. O intervalo de n investigado vai até 10^5; extrapolações para escalas maiores assumem que o regime assintótico já foi atingido no domínio medido. Os resultados devem ser lidos como evidência de tendências de complexidade, não como benchmarks absolutos de desempenho.

Trabalhos Futuros

Uma continuação natural seria reimplementar o `find` de Tarjan de forma iterativa em dois passos (subida até a raiz seguida de compressão), eliminando o overhead de chamada recursiva e validando experimentalmente se o ganho de Tarjan sobre Rank se torna mais expressivo. Outras variantes de compressão, como *path halving* (cada nó passa a apontar para o avô), também merecem comparação. Por fim, ampliar a escala para $n = 10^7$ e variar a densidade m/n permitiria investigar como a topologia afeta o termo constante oculto em $O(\alpha(n))$.

Referências

[1] Tarjan, R. E. (1975). Efficiency of a Good But Not Linear Set Union Algorithm. *Journal of the ACM*, 22(2):215–225.

[2] Tarjan, R. E. and van Leeuwen, J. (1984). Worst-case Analysis of Set Union Algorithms. *Journal of the ACM*, 31(2):245–281.

[3] Tarjan, R. E. (1985). Amortized Computational Complexity. *SIAM Journal on Algebraic and Discrete Methods*, 6(2):306–318.

[4] Kruskal, J. B. (1956). On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem. *Proceedings of the American Mathematical Society*, 7(1):48–50.

[5] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. 3rd edition. MIT Press.

[6] Sedgewick, R. and Wayne, K. (2011). *Algorithms*. 4th edition. Addison-Wesley.